ActInf ModelStream 009.1 ~ Aswin Paul "On efficient computation in active

ModelStream | Aswin Paul | 1:00:12

Hello and welcome everyone. It's July 15th, 2023. We're here in Active Inference Model Stream number 9.1 with Aspen Paul. Today we're going to have a presentation and a discussion on efficient computation in Active Inference. So if you're watching along live, please feel free to add comments or questions in the chat. Otherwise, Aspen, thanks so much for joining today. Really looking forward to your talk. Thank you, Daniel. Thank you so much. So as mentioned, today I'm here to talk about efficient computation in Active Inference and yeah, let's get started. So we're all familiar with this idea of the free energy principle, which is also known as active inference, right? So the central concept is that an agent minimizes entropy of its observation. To maintain homeostasis or survive in its environment. And here the entropy is defined in the information theoretic sense, right? So if an observation is highly probabilistic, that is less entropic or less surprising because it was high probability and we were expecting it. And that's the idea. Then that's the base that we build this framework of active inference on. And this idea of Markov blanket, it gives us a systematic way of surprising or systematic way of separating an agent from its environment and model purposeful behavior, right? So let's focus on this idea of minimizing entropy. So how does an agent minimize entropy or know which observation is highly probabilistic and vice versa? So that is by maintaining a generative model. And the generative model is basically a toy model of the environment, which the agent builds in its brain. And that is built using only the observation that it get from the environment. So it has no access to the real states or the hidden states of the environment. It's building the toy model. And given this toy model, it has scope or ability to compute the probability of an observation and hence try to minimize the entropy, right? So that's the idea, but it has a problem of, um, cause of dimension dimensionality, like given a generative model, it may not be possible always, uh, to calculate or marginalize the probability of observations out of it, because the state space can quickly become intractable. But the idea is that you define an upper bound on the surprise using Jensen's inequality. And you may also define a new term called Q, which is the hidden, um, belief or the belief about the hidden states. And this Q is going to be the focus of decision-making, right? So if you have a noisy Q and you have no idea what's in the environment, then you can't make or hope to make decisions to control that environment. And it is this belief about the hidden states that you use and that becomes useful to take decisions. And this whole quantity is of course called the free energy and the variation free energy, uh, F can be interpreted in multiple ways. So the first or the most common is the machine learning way of how it is, um, trying to minimize the complexity of the model, uh, at the same time trying to maximize the accuracy of it. So that's the machine learning interpretation of minimizing various free energy. You may also try to, um, interpret free energy in the physics term where you at the same time, try to minimize the energy of your model, um, but at the same time trying to maximize the entropy, but the focus of today, uh, or decision-making is always on this belief that you have, uh, after you do the perception in active

inference. So how do you do vanilla decision-making or what is the most discussed idea of decision-making and classical active inference? Uh, so if you, uh, are in an environment, uh, and if you're an agent who's trying to make decisions, then you have a, uh, space of available actions, right? So in this toy model, you have three available actions, uh, run, jump, or stay, uh, and given these actions, you can hope to define a policy by small pie, which is a sequence of actions in time, uh, and capital T is the time horizon of planning or that's the length of your, um, policy. And you have a policy space with many such policies, small pie. So given this bigger small space, uh, you can attempt to evaluate the expected free energy, uh, based on the beliefs that you have accumulated. So here you're not minimizing anything. You already have a belief from variation free energy, uh, and you, you are just calculating or evaluating the expected free energy corresponding to many, uh, small policies that you can, uh, define. And, uh, after you evaluate it for all policies, then you know, which is the optimal policy to take. And that's the classical, uh, active inference idea of decision-making, right? Uh, and this expected free energy is really useful in the sense that it is, um, goal directed. So the risk term is, um, goal directed, and then you have this expected ambiguity term that forces you to explore as well. But there's a problem that this policy space can quickly turn interactable. And that has always stayed, uh, as a bottleneck, uh, in scaling active inference, uh, to, um, commonly seen environments, but let's see how it becomes quickly interactable. So how many policies can be defined, say for a time horizon of 15, right? So if you are playing, say, super Mario, uh, you might, might want to plan at least say 10 times to the head. So the first policy could be, um, the same action run, um, stacked for 15 time steps. And you can, um, basically define several combinations of such, uh, actions. And the policy space is simply, um, interactable. It becomes, uh, too huge for you to, uh, evaluate the expected free energy for all such policies. And in a stochastic problem setting where, um, the environment itself is noisy, you don't really have a, uh, method to choose a subset of this policy space and do classical active inference. And this is clearly a computational interactable problem. And that's why in literature, we always see small grids or small environments, uh, when you discuss decision making and active inference, but recently, uh, a new idea was proposed, which is called the sophisticated inference. And in sophisticated inference, it is not really the policy space. You are actually, um, real time trying to think what to do. So if you have a belief, then you are trying to, um, evaluate the actions based on that. So here we don't have a sequence of policies or, um, things that becomes intractable. Uh, here we are doing basically research in the sense that you are trying to evaluate expected free energy of, um, this, uh, joint distribution of actions and, uh, observations. And to evaluate the expected free energy at some time, uh, small t, you will also need the expected free energy, uh, at the next time step. And to evaluate that you will need the expected free energy of time t plus two. And this basically becomes a research that rolls out in time. And, uh, it's a, it's a recursive relation and here it's fundamentally different from the policy space that we saw in the last slide. Right. And is it computationally better in the sense that if you have to plan, say 10 time steps ahead, uh, comprehensively, uh, in classical active inference, we saw that the policy spaces, um, the cardinality of the action space, uh, raised to T.

So that is the computational bottleneck. If you have to, uh, consider all possibilities, but in sophisticated inference, it's even worse because, uh, it, you are considering a combination of state and actions. So it is computationally worse. In fact, uh, but a solution was proposed in the sophisticated inference paper that we can do pruning of this research that we can avoid some, uh, states and actions, uh, when you do this research and that becomes computationally very tractable. So let's see how pruning, uh, works in sophisticated inference. So this grid was discussed in the original sophisticated inference paper and say for this grid, uh, given that you have this kind of, uh, prior preference distribution. So this white square, which is the gold state is the most preferred state and you have, uh, um, uniformly decreasing preference for states that is far away from that, uh, gold state. Then basically if you observe yourself at some observation at time T, uh, then basically what you're doing is you're considering, uh, the consequence of, uh, available actions, uh, from that observations. And basically you can use, uh, the projection of your belief, uh, and set a threshold for those beliefs to kind of maybe ignore some actions and ignore some observations. And what you will find is that it is, it's no longer a tree search. It's a subset of tree search and it's computationally way, uh, efficient than doing the whole tree search, right? So here you have avoided a lot of combinations and this becomes a computationally tractable problem. This is because you have decided to avoid some actions and, um, observations. So you are essentially compromising on that possibilities which might give you a higher reward or it might be more optimal than the consequence of this, uh, partial tree search, but this works. So this simulation was presented in the paper and it works. The agent, uh, kind of learns to do, uh, what needs to be done if it plans in this forward direction of planning, uh, in, in a pruned way, but, and, and basically you can computationally show that even for a small, um, search threshold, um, you can, uh, drastically decrease the computational complexity. So if you decide to do the whole tree search, um, with the planning depth, uh, the computational time is exponential and quickly, you can't do much about it then, but if you decide a search threshold, which is even very small, uh, you can see that this problem becomes computationally tractable. Uh, and this demo, uh, on sophisticated inference is available in my version of my MDP and it's going to be, uh, integrated to the original by MDP soon. Um, so, but the key point is that the pruning of the tree search requires us a, a well-informed prior preference like this in the sense that here the agent is aware of, uh, how desirable are neighboring states. It not only knows about the final goal state, it also knows about its, uh, neighboring state and given such a prior preference, we can see that for a planning depth of three, uh, the agent is getting stuck basically in this, uh, local maxima of prior preference, but with, uh, sufficient planning depth, um, it is, it is able to, um, overcome this barrier and reach the goal state,

uh, in, in this great problem. And the question is that what if the agent only knows this final state, it has no other, uh, knowledge about what to do? And in this case, what does the agent do? So this is the problem that we are trying to address. And in the absence of a, a meaningful prior preference like this, the agent has basically no way of, uh, reaching the goal state, um, other than by random exploration. It, it has no way of planning because it, it, it cannot do, uh, planning eight time

steps ahead because it's computationally interactable. Uh, if, if it chooses, uh, to do the full tree search, right. And so if, so for a grid like this, what do you do if you get a sparse prior preference, which is not well informed and, uh, as highlighted in the previous slide, your research is now blind. Uh, you have no way of pruning the research and you have to do the full tree search in, in this scenario. So as you might have thought, um, there are two solutions to this. Either you have to find out a way to do the full depth planning, uh, uh, with this spiral, uh, this, with this sparse prior preference, or you have to learn a meaningful prior preference, which will enable you to do, uh, this pruned research. So we are going to discuss these two solutions, uh, in this, uh, presentation, uh, for a given scenario like this. So the first solution, uh, to do full tree search is basically using dynamic programming and, uh, dynamic programming is a well-known idea in, uh, operation research and industrial engineering and many, many branches of engineering. And the basic idea is that you solve the sub parts of a bigger problem first, and then later try to integrate, uh, the solutions of these sub problems, uh, to do, uh, optimal decision making. So in this scenario, imagine that, um, you're trying to plan, uh, for the last action. So earlier we started from the first action. We started from the present and try to predict what is happening in the future. And in, in, so your direction of planning was basically forward in time, but imagine that you are trying to plan only for the last time step where you are just near the gold state, uh, and right going to that gold state, right? So you are trying to make a decision for that last time step, which is capital T minus one and your projections, uh, to the next time step or the last gold state will, uh, can be done because you have access to this model of the world, uh, using this, uh, transition dynamics or the B matrix and active inference. So for this single time step, which is a sub problem, uh, you are actually evaluating a table, uh, of expected free energy that tells you, if you are in say this observation, what is to be done, uh, which is for the last time step. And basically, uh, you can do this in state space or observation space. So this can be done, uh, using the A matrix and B matrix together, uh, where you can do planning either way. Right? So the question is that if you know what to do, uh, in the last time steps, then you might be thinking, how do I know that I'm in the last time step? Uh, it is all about imagining it. You have, you have, you have, you have, you have, you have, you have, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, um, what you will do if you are in time capital T minus one which is the last time step for your planning horizon and if you are in that time step what do I do so this table represents all such scenarios that if I am say at observation three at time t minus one what do I do and this quantity here I'm only considering the risk term or the purposeful term and this term represents that policy and what if I do this backwards till time t minus one so if I am if I know what to do at time capital T minus one then this table can inform what to do at capital T minus two so rather than planning forward in time what I'm doing is basically stacking many tables together just by fixing a capital facility of planning and given that I have all such stacked tables then

basically what I can do is use them to take decisions forward in time and what we have observed is that this idea works that I can calculate the expected free energy backwards step by step considering them as sub problems and the fundamental difference is that in sophisticated inference to calculate the expected free energy at time small t you don't know what is the expected free energy at time t plus one so this becomes a research so that you have to calculate this first and to calculate for t plus one you need t plus two and so on but here because you're calculating it backwards in time you already know what is the expected free energy for t plus one and you base it's basically the same equation it's just that you're doing it backwards in time and pictorially in sophisticated inference you are trying to do a tree search but in the dynamic programming algorithm you are you doing your planning backwards using tables and given your planning horizon is sufficient enough for a problem what we have seen is that the agent will be able to take optimal actions forward in time so in the paper we are also proposing one algorithm for sequential form dps using this backward planning in time and we were able to scale up simulations for grid spaces which was previously interactable without neural networks so that was the first solution so the second solution is that so in the first solution it was fixed that you only get the sparse prior preference you don't get any other information you only get your information about the final goal state and all you get is the model of the environment which you learn and you basically have to take decisions but the second solution of course is that if you are allowed you can attempt to learn a prior preference which is meaningful like the one we saw in the previous slides which has the information about the other states also but how do you learn it right so there is a seminal work from optimal control literature that talks about efficient computation of optimal actions and in that work there is a quantity similar to our prior preference which is called the desirability function so for example in this grid world here darker colors are more preferred so if this crosses your final gold state what the agent in this paper is trying to do or the said learning method in this paper is trying to do is learn this desirability function as optimally as possible and what has been shown in this paper is that if you try to learn the desirability function using a particular learning rule it is computationally far more efficient than even Q learning which is a well-known reinforcement learning algorithm so Q learning is a well-known computationally optimal algorithm but in this paper that particular learning rule for learning this kind of a desirability function is way faster and this approximate error represents how different it is from the optimal desirability function so this desirability function is nothing but our prior preference and as mentioned learning said is what was of magnitude efficient than learning the Q function in Q learning so this is the particular learning rule depending upon the reward that it gets from the environment and in

this particular grid world environment we are only giving it reward at the last step which is similar to

the sparse print preference the agent basically gets no reward until it reaches that final goal state and

with this learning rule which which has a parameter eta that controls how fast or slow this learning happens so

basically we try to study the effect of this learning parameter eta but what we observed is that it's

very robust it can learn the prior preference reliably even with variable values of this learning parameter

and what we see is that the agent is able to learn a meaningful prior preference over time very fast and using such a

meaningful prior preference then the agent don't have to plan a lot it can manage optimal behavior or purposeful

behavior with very low time horizons of planning and given these two solutions we could scale up active inference algorithm for decision making

and talking about the computational efficiency we saw that the dynamic programming method could plan 30 time steps into the future

with only say a computational complexity of 1000 when compared to say 10 to the power 68 for sophisticated inference and the

second method which learns the prior preference which we call active inference and it only needs to plan one time step ahead so it is way more

computationally efficient in that sense but it has to learn so in the dpfe method we are not learning we are using the

sparse prior preference and doing the whole depth of planning and in the other method we are letting the agent learn this prior preference

but then we can save planning a lot because it knows a lot of what to do in time right so graphically we can see that the dpfe method is really computationally efficient and

when when plotted against time in the ai ft equal to one method it is basically computationally way cheaper and yeah so it scales very fascinatingly well with higher and higher complexity of the environment so we tested this these methods in

um very huge state spaces like say 900 states compared to um state spaces which has dimension of say 5 or 10 in the usually seen active inference literature um so i want to kind of emphasize that we are not using any neural networks here we are using um explainable active inference agents uh doing all um necessary matrix multiplication so that we have access and um explainable

um explainability to every computation that is taking place in this in these algorithms and when tested on these grids first we validated this on a smaller grid with 100 states and we observed is that when compared to benchmark reinforcement learning algorithms like q learning and dynaq

uh dynaq is a model based reinforcement learning algorithm and we compared our newly proposed agents which is dpfe and aif and we saw really um good performance uh and aif agent is slightly worse and that's because it's only planning one time step ahead right but

uh the dpfe agent who does full time planning it's performing as good as benchmark reinforcement learning algorithms and we tested it with um bigger and bigger grids

and when uh we introduced stochasticity in the goal state in the sense that when the agent uh had to uh navigate to um stochastic goal state so we changed the goal states at every 10 episodes and what we

observed is that the dynaq took more time than our dpfe agent to uh kind of recover and do well so dpfe agent in this stochastic environment performed really well even better than the dynaq agent

and you could also see that the ai agent is recovering um faster but not as good as the other agents so basically that was the result in the paper and the methods so yeah thank you for listening and i'm open for discussions

all right awesome wow very cool thank you okay well just as we

start the discussion um if anyone wants to add anything first just how did you come to work on this

project uh were you studying active inference and you came to this question as being interesting or were you working on the planning and came to active inference as a method um yeah so a little bit background about myself so i studied physics both in my undergrad and postgrad and towards the end of my postgrad

i got interested in things like game theory and reinforcement learning and i joined uh for a joint phd

with professor adeel and professor manoj and so professor manoj is a control theory person and adeel is a neuroscientist and in the beginning of my phd i started reading active inference literature and i wanted to implement that in uh problems and i was always fascinated with uh explainable uh active inference

uh about this idea of expected free energy trying to minimize risk as well as expected ambiguity which

also not just making the agents work but also being able to tell how they work is what fascinated me in active inference and initially i tried to implement them and there is a conference paper uh which we published in which we

uh we compared uh it for a similar grid world task and that then i faced this problem of scaling active inference and then i started working on sophisticated inference um yeah so basically uh these methods came out of the need that i wanted to scale them up and i always had was skeptical in using um deep active inference because um i didn't want to use neural networks to do uh planning because deep reinforcement learning in itself is a huge field and um if you if you're about just scaling active inference then maybe just do deep reinforcement learning and that's what i thought yeah so that's basically the background and yeah that's how it came about all right i'll go first to a question in the live chat and we'll just

probably discuss various aspects because there was a lot in your presentation so ml don writes i wonder when it comes to computing the expected free energy over a time horizon what kind of mean

field approximation was used to factorize q of s so

so

so i assume that the question is about um the expected free energy in dpfe and to calculate this basically uh so in my uh simulations i use belief propagation for this um belief q and um you can also also use variation message passing or marginal message parking that's not a problem but

so once you have this belief q um what do you do right so

yeah so

when i say uh you imagine um q for a time then what i use mostly is one hot vector so

to take decisions you use the um belief that you get out of your perception step in active inference

uh but for uh imagining these are hard tables where they are one hot vector so

uh given say you in your generative model uh has 10 states uh your the cues that you use are um

precise cues for planning but for decision making you use the imprecise imprecise uh main field approximation cue that you get from the perception step so i don't know if that answers the question um but maybe i i also want to think um i also need to think about more about the approximations that may be different

cool yeah they can they can write more if they want um let's talk a little bit more generally about preference learning so in the context of the active inference generative model we have a mediating between observations and hidden states and learning makes a lot of sense it's about learning the mapping between observations and hidden states of the world and then we have the learning learning the consequences of action and how things change through time and preference learning is learning on c c and you've highlighted that that this is a very um you know interesting variable to learn and i'm curious how does how do we learn to learn the right thing how do we know that we're learning in adaptive preference and then how does that preference learning reduce cognitive overhead or computational complexity yeah great um so if you are trying to learn a prior preference then that assumes that there is something to chase or there is something to maximize like a reward

so in this say uh in a reinforcement learning setting there is clear reward that's coming from the environment that you're trying to maximize and say for this grid you get that reward only in the last step that which is that that's that's what makes this problem hard so if you are getting rewards at every time step then basically you know what to do that i just have to pursue that reward at every time step right but here you might have to take say uh 15 time steps ahead uh to get that one reward and that basically is hard to do so here um if you are trying because i'm trying to learn a prior preference um there should be a reward structure that uh that exists in the environment if the environment doesn't care about what i do or if i can't define what is good or bad uh then there is no meaning in learning the prior preference right so here um the reward is what controls the learning of this prior preference and the good thing is that even if only i get the reward at the last time step um i have my b made matrices and i have experience of transitioning from different states um which i reached this final goal state and this algorithm that we are using or the learning rule that we are using

uh is precisely learning a similar thing in uh optimal control that paper i introduced which is called the desirability function and given that you have this desirability function so imagine that maybe i am starting from this state uh then i just have to look at my nearest neighbors to take a decision um if the near a state at um state below this state is more preferred then i only have to plan one times to head ahead and that's the optimal decision i have to take so learning this prior preference reduces the cognitive load in the sense that i am only now looking at nearest neighbors i don't have to plan all the way up to this final goal state and i will learn this as efficiently as possible because one it was it's a guaranteed algorithm we tested its robustness uh and it is also informed by the reward that i get from the environment and if yeah so if there is some way uh to kind of define what is preferred uh then you are guaranteed to learn a prior preference that's meaningful by this algorithm so if only nearest neighbors and one time step inference is being used then what prevents this kind of preference learning agent from being stuck in a local optima um yeah so there won't be

local optimus here that's the point like um why would there be a local optima uh if i am learning it using rewards uh yeah so if if there is local optima then it will be getting stuck in the local optima but there won't be one uh if you're learning it's this way because you're learning it from your own experience and uh how you caught the reward at time uh sometime and what we observed is that uh it's very um gradual and there is no glitches or local optimus when you learn such a practical so it's kind of backfilling a preference so that there can be a smooth path yeah to the distal goal yes so it in the animation kind of makes it look like a backfilling but um you're actually learning it from your experience forward in time right so i observed a reward when i transition from this state to this state so this must be good uh then in the in the next time step i say okay this state is responsible for taking me there so this might be good also but not as good as the other one so it looks like it's learning backwards but it's actually learning from the real um t2t plus one x uh experience
okay yeah so in that way um you can't have local maximus because um
um yeah this is the learning rule that does that i don't know how to answer it more systematically
yeah um what what real world situations or or what real life situations do you kind of see this kind of preference learning happening in yeah great question so so there is one uh paper that came out from our lab uh where neurons are laying learning the game of pong um so there was so if the paper is called dished brain uh and the device is called dished brain and what they have accomplished is that they managed to culture neurons uh on a silicon chip and they um gave it a good uh feedback signal when it successfully tackled the ball or uh played the game well and gave it a shock uh when um it made mistakes and what we saw is that uh in the paper what we see is that uh it learns to play the game over time right uh and in in such a scenario imagine that if i encounter say a positive thing in the world then i might associate um what is good with the states i observed in the past and so on right so i learned um step by step what is good or bad and that's a reasonable hypothesis to make so if um if i want to say um get fitter uh then i might associate going to gym um
um as a good thing then i might also associate walking to the gym as a good thing then i might associate uh say wearing my shoes as a good thing so um learning prior preference that way makes sense i would say but yeah this is a computational um solution to the problem of curse of dimensionality but testing this on the real world is definitely uh what we should do and um yeah i also look forward to doing that okay let me let me try to run a real world situation by you and see if this if this connects um so we want to we want to turn in we're rewarded by turning in an essay that gets a high grade and so there's many steps between starting the idea of the essay and seeing that preferred sparse outcome which is the grade um and we we do all kinds of things and then we learn okay well um this one where i did get a good grade it was well formatted and then that kind now you sort of expand the umbrella of your preference out and then you go what brought me to getting it well formatted well i did it on time and then you kind of work all the way to the ideation phase so that in the future you can do one step inference yeah exactly and just take it step by step as a skill instead of needing to do a 15 time step inference over all the possible tree structures so you've kind of used your um embodied learnings to simplify the structure of the problem exactly yeah

and can we look at the computational complexities um again of the of all them
so here um cif stands for the classical active inference and to do the full horizon of planning
um it's around 10 to the power 18 so this is for this particular grid example with um 100 states
uh and four available actions uh so this is for a special case and i just wanted to put numbers to
put things in perspective so for this particular example if you are trying to do it with the policy
space thing um you will have to do um 10 to the power 18 computations for one step of planning or in one
instance in sophisticated inference it's even worse because uh there you the state space also matters
uh and that won't work so this third line was supposed to kind of show that even for a time um
horizon of two it's really hard to do sophisticated inference if you have to do the full planning but
with dynamic programming when you're planning backwards um you can attempt to do the full horizon of
planning um and that's only a thousand computations uh but with learning prior preference it's even low
uh when you do it just one time step ahead wow this is quite um quite stark and in the branching time
active inference work in a previous model stream we also saw some computational complexity estimates but
i think these are really clear um first thing it made me think of is no one said sophistication was cheap
i mean it's it's astronomical how costly it is for um maybe even only at two or three or four or five
potentially starting to get up to uh so it's increasing the the complexity of planning um radically
so it's very um interesting um pedagogically as we think about like as we imagine active inference
intelligence architectures but this makes it pretty clear that it's not something that you can just
enumerate over and so i think it's very um very creative and important what you've done by
connecting this to principled methods of computational complexity reduction rather than potentially
effective but ad hoc methods of complexity reduction like neural networks which might work when they work
but then once those start to get bloated now you have no principles and no efficacy exactly yeah
and sophisticated inference i must note that has its own benefits when compared to dynamic programming
in the sense that when you're planning forward you are taking into consideration all the possibilities
right but when you're planning backwards say for examples like queue based exploration um like if you
have to go somewhere get a queue and then navigate to the goal state then planning backwards might not work so this is what this was one feedback i got when i
i discussed uh the work with um um some people so um that's a thing to note about dynamic programming but the other
other place where you learn prior preference i believe it's not a problem but yeah dynamic programming is computationally cheap but it also come with its own limitations i must note here so just to restate that in dynamic
programming with a bellman optimality we're solving backwards and so it's kind of like the checkmate is
what we want and so now we're working backwards to the present but we end up not exploring counterfactual endpoints we don't work back to the present to endpoints that we're not interested in
exactly so that's why it's so ruthless but on the other hand it's a much more constrained um
search yes so yeah so what i'm uh thinking about these days is that we should also consider uh
combinations of these two uh where i can afford to kind of ignore such counterfactual things um there i can

do dynamic programming uh and maybe i can do as two steps uh of planning so if it's a cube based exploration
say reaching q uh is one task and from q to the reward is the other task so i can separate these two
and use dynamic programming for them uh and that's computationally cheaper but also kind of preserves
this idea of um having to do it yeah but these are all future things i'm thinking about cool i'll ask
another question from the chat alex wrote do you have ideas or developments on nested models where
different scales could have different um one time step ahead and speed of execution so how does this model
play out in nested models and how do we think about this forwards and backwards in time in the speed of
execution and nested models okay so i might need a little more context here like um when you say nested
models um do you mean hierarchical models yeah yeah so maybe um something like uh there is a observation i get
on a fast pace uh and i do some inference on that but then i use this inference uh to kind of uh do or maybe
uh and that basically this inference becomes the observation for the next state and that's what
nested models mean right yes so i might have to actually think about that in terms of a task
um and then think about it but honestly i i haven't um thought how this might work in that context
uh but say for a task um like uh so in terms of navigation if you think about it you can think about
um say rooms environment so that's where uh that's one of the examples i can think about uh where we can
apply this so imagine maybe you have a collection of rooms uh and as an agent you have to first figure
out which room to go and then you have to navigate inside that room and basically uh you can do uh
inference in two stages or decision making in two stages uh and you have you will have to separate
your decisions in in those stages inside the room you can do say dynamic programming to navigate your optimal
uh path but you will always have to kind of have two stages of um decision making and um maybe different
methods work better at different stages uh but this would be uh better in you know i mean this discussion
would be better to uh better in in a well uh thought out task i would say i i don't have a answer
um that might be uh fitting for everything but yeah yeah we've seen almost exactly that kind of
hierarchical simultaneous localization and mapping slam in a robotics case there have been active
inference models on that um yeah would it be possible to sort of do um one of these methods
methods at one level of the nested model and then have another computational method applied to another
um or is like if you wanted the advantages of one in one place like can you mix and match with these
different methods even within one simulation yeah i i definitely think that's possible um but
yeah we'll have to maybe try so all my methods are um one stage it's not a hierarchical uh model and but
i i strongly believe that it would also work in an estate model where you can have um two methods of
decision making working together in say for a room example i just mentioned well could you go to the slide on z
learning yeah cool yeah so i noticed um here and in the great paper that you had several total of
citations yes and so and this introduction of the z learning is is a novelty with respect to the active
inference field so could you explain a little more what is the z and what is it that enables such a a rapid um

improvement in the z with respect to the q q yeah okay so uh to give some context uh context about this paper

um it's it talks about a linear method of decision making uh in a particular class of mdp so given that you are having a mark out decision process where your actions can be uh based on states and not actions so when you think about actions in a say in for example a grid world task you think about left right and north south right so if i take a north then that has a consequence of uh consequence on state space but in this paper um they are introducing a class of mdp where decision is itself in terms of states so if i am saying state s1 my decision will be uh depending upon the other states so my decision will be based on the other states i want to be in the next time step so uh it's a redefinition of decision making uh in terms of state space uh in terms of state space rather than decisions being something else like left right and down up so given that such an mdp exists where i can take decisions in terms of states they have showed that computationally uh you can take uh decisions in linear complexity uh however big the problems so for that to so to enable decision making in terms of states you should have a sense of what is good and bad uh for the states so in this grid word example uh you have a desirability function which is c so c is the desirability function uh which talks about how desirable is the state uh and if i have a c function uh then what they have showed is that uh i can take decisions with linear computational complexity for this particular class of mdp so if my mdp allows me to take decisions in terms of states uh it's linearly uh it's only linear complexity for that thing so this graph is basically comparing how you can learn desirability uh better uh or faster right so q learning if you're familiar with q learning it's basically a table based method uh where you have um desirability of actions given a state so given a state you know what to do that's basically the q matrix but in terms of c uh or uh the c learning method it's only about states uh you're only learning how desirable a state is there is no concept of uh actions here and this is exactly what our prior preference uh is in active inference where it is a distribution that quantifies how desirable or non-desirable states are right so uh given that they have shown that you have you can learn this c matrix faster and that's optimal and it's way faster than even q learning then i thought okay why not try to learn c the same way as c is learned in this paper and um using this say so there is a similar learning rule for learning c which is called set learning in that paper and when i attempted to learn c what i've seen is that it learns really fast a useful prior preference that lets me take decisions or let the active inference agent take decisions um only by say one time step of planning and yeah so basically that's the story so um this idea of c being easily learnable uh is in that paper uh the terror of paper okay let me try to um restate that since i think it's a very interesting augmentation of active inference so we're going to be learning c for all the reasons we discussed earlier we're going to learn c analogously to how todorov presented z learning and in the z learning instead of learning um for example updated posterior probabilities on actions and then using actions to navigate amongst states that emit observations yeah we're going to kind of bake the action into the states so that really we're learning transitions among states directly yeah and and to connect this to the free energy principle and and how it is playing out with with active inference here um c is not just our desirability function

that's one way of thinking about it that's why we call it preference but also c is our expectation

and so that is what allows us to on one hand use the language familiar to reward and preference learning

like the agent is ending up where it likes to be but also the expectation based um definition of c

these are the same thing allows us to talk about that as a path of least action or as the most likely

outcome or the least surprising outcome and because we've defined it what we want as the least surprising

outcome then we can use variational free energy to bounce surprise whereas you can't use simply a

variational method to bound or even necessarily approximate reward itself but if you say

i prefer what i expect and what i expect reduces my surprise and i'm going to bounce surprise then you get

both that sort of behavioral reward seeking couched in a surprise bounding surprise minimizing physics framework

to make this is to

first of all address the limitations of dynamic programming so if if you have say a queue to

explore in this grade first and that's more optimal i want to see how the expected ambiguity

term in the expected free energy is useful and should be made use in the dynamic programming

strictly in that sense but more generally i'm also looking at other ways of making decisions in

active influence like there are works from professor isamura from cbs ricken he talks about how it's how

neural networks are doing active inference and basically decision making there is really

more efficient in the sense that it's like q learning so in there he's making clever use of

the variation free energy to learn good state action mappings and it's it's a very drastic

change to what we are used to in terms of expected free energy so there is no concept of expected free

energy in in that work it's all about learning what is good and bad directly from variation free energy

so i i find that also fascinating i want to kind of explore that more and see how decision making

uh in this way is better or worse or should be thought about should we even reconsider ways of

decision making because active inference only talks about variation free energy and that's the central

tenet everything else is uh your interpretation of that right so yeah that's another direction i want to go

interesting way to say it um definitely the variational free energy which is a functional of our variational

um distribution q and the data y the variational free energy is kind of like the real-time homeostasis

like yeah how are things making sense given what i believe and the incoming data and then to extend

that kind of a sense-making framework into decision making we've seen a lot of different methods expected

free energy is um a common one but for example there's been free energy of the expected future

f-e-e-e-f and there's other constructions that have different methods um and then you pointed also to

professor samura's work with um the sort of uh relationship between the variational free energy

on the base graph and the loss function in a neural network and all those relationships that's very exciting work too

um i guess kind of in closing just as a last question or thought you're coming close to the end of your

phd so just in the time that you've been a phd student how have you seen active inference develop or what what

what what feels different to you today near the end than when you were yeah fresh-eyed and excited several years ago

yeah that's that's a really great question and uh i am also very excited of how the field has evolved

uh and frankly i started from this reinforcement learning background and the physics background and

uh when i started reading it was only say one or two papers and i didn't not quite understand much of it

uh it was only when i started uh implementing it using cars um math lab scripts i kind of understood okay

this makes sense and i like it and but within uh say one or two years i saw a lot of papers coming in

from different directions and people also starting to use neural networks and all this scaling up came

uh and i i at some point i also questioned the need of active inference uh because if you have deep

reinforcement learning can which can do many things then why deep active inference uh and that's the

reason why i did not get into that but i still find it fascinating i want to kind of understand deep

active inference more than what i know now uh but i've seen the field growing like anything in in two or

three years and many cohorts of people starting to work and uh in no time it was a seriously taken field

other than a field with say two papers with nobody actually knowing what it is so it's really exciting

yeah so i really look forward how the field evolves in time and also what i can do after my phd and so on

cool forward in time backward in time what we prefer what we expect yes

any other comments or anything else you want to add yeah so please let me know uh what you think about

the paper and uh what you think about these ideas feel free to let me know i i really look forward to the

the feedback and yeah thank you so much for this opportunity daniel and thank you for your time

it was amazing i hope people um check out the paper and get in touch and replicate the code and

and take it their own directions thank you see you next time see you next time thank you so much bye

have a good day bye

um

Thank you.